Semantic Log Analytics for UAV Flight Logs

Treating flight logs as a database problem

Sumit Kumar (22CS30056)

Advanced Database Management Systems, term project

github.com/SumitKumar-17/UAVLog

Motivation and problem statement

Why flight logs are hard

- High-rate time series: 300 PX4 flights hold **70.7 M** samples
- Free text: warnings, failure flags, pilot notes (3,401 events)
- Vendor dialects: PX4 and ArduPilot name the same quantity differently
- Tools plot **one log at a time**; people ask *"which flights lost a motor?"*

Problem statement and research questions

- **RQ1** How should high-rate telemetry be stored, and at what cost?
- **RQ2** Which vector index and which filtering strategy retrieve events best?
- **RQ3** Can a model write safe, correct SQL, and what does a semantic layer add?
- **RQ4** Does the pipeline survive failure and scale to hundreds of thousands of logs?

Background: the database building blocks

TimescaleDB (time series)

- **Hypertable**: one logical table split into time *chunks*
- **Columnstore**: old chunks stored column-wise and compressed, grouped by `segmentby`
- **Continuous aggregate**: incrementally maintained rollup (1-second min/max/mean)

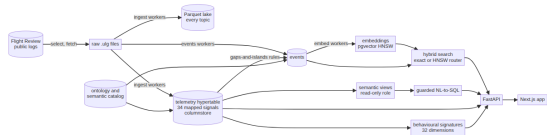
pgvector and pgvectorscale (similarity)

- **HNSW**: layered proximity graph, fast approximate search
- **IVFFlat**: cluster lists, probe the nearest few
- **DiskANN**: compressed graph on disk (pgvectorscale)
- Known pitfall: **filters apply after the index scan**, so filtered queries can under-return

The gap in existing work and the proposed architecture

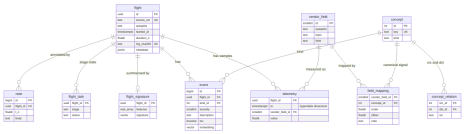
Gap

- Log viewers: one flight, no semantics, no text query
- Time-series DBs: no ontology or vendor mapping
- NL-to-SQL demos: rarely guarded or measured
- To our knowledge no published system combines all three over PX4 logs



One PostgreSQL instance holds the time series, ontology, events and embeddings.

Core contribution 1: schema design



```
Migration 0001: the telemetry hypertable
79      # ---- Telemetry: only ontology-mapped ("hot") fields; full-rate raw stays in Parquet ----
80      op.execute("""
81      CREATE TABLE telemetry (
82          flight_id       uuid        NOT NULL REFERENCES flight(id) ON DELETE CASCADE,
83          ts              timestamptz NOT NULL,
84          vendor_field_id smallint NOT NULL REFERENCES vendor_field(id),
85          value           double precision NOT NULL
86      )""")
87      op.execute("""
88      SELECT create_hypertable('telemetry', by_range('ts', INTERVAL '7 days'))
89      """)
90      op.execute("CREATE INDEX telemetry_flight_field_ts ON telemetry (flight_id, vendor_field_id, ts DESC)")
91
92      # ---- Semantic view: telemetry expressed in canonical concepts ----
93      op.execute("""
94      CREATE VIEW canonical_telemetry AS
```

Design decisions

- Narrow long format: a new signal adds rows, never columns
- `telemetry` hypertable, 7-day chunks, 70.7 M rows
- `vendor_field` and `field_mapping` carry scale, offset and a note
- `event` holds text, a `vector(1536)` embedding and a generated `tsvector`

Core contribution 2: the semantic layer

Ontology

- **63 concepts, 55 relations**: 34 signals, 7 subsystems, 7 failure modes, 15 event kinds
- Signal → subsystem → failure mode → event, walked with recursive CTEs
- Queries say `battery.voltage`, not the vendor field name

ArduPilot: a wrong mapping is worse than a gap

- Mapped: attitude, GPS, position, velocity, battery
- Not mapped: vibration, motors, EKF, barometric altitude, mode

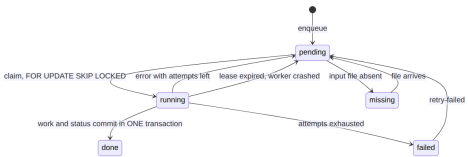
A documented semantic view: events in canonical terms

```
46    CREATE VIEW semantic.events AS
47    SELECT e.id AS event_id, e.flight_id, c.key AS kind,
48           e.severity, CASE e.severity WHEN 2 THEN 'critical' WHEN 1 THEN 'warning' ELSE 'info' END AS severity_label,
49           extract(epoch FROM e.ts_start - f.started_at) AS start_s, extract(epoch FROM e.ts_end - f.started_at) AS end_s,
50           e.description, e.source, fm.key AS failure_mode
51    FROM event e JOIN concept c ON c.id = e.kind_id JOIN flight f ON f.id = e.flight_id
52    LEFT JOIN concept_relation r ON r.src_id = c.id AND r.rel = 'indicates'
53    LEFT JOIN concept fm ON fm.id = r.dst_id AND fm.kind = 'failure_mode'"""
```

ArduPilot mappings carry a scale, an offset and a semantic note

```
142  ARDUPILOT_FIELDS = [
143      ("ATTITUDE", "roll", "rad", "attitude.roll", RAD2DEG, 0, "radians to degrees"),
144      ("ATTITUDE", "pitch", "rad", "attitude.pitch", RAD2DEG, 0, "radians to degrees"),
145      ("ATTITUDE", "yaw", "rad", "attitude.yaw", RAD2DEG, 0, "radians to degrees, range -180 to 180"),
146      ("GPS_RAW_INT", "lat", "degE7", "gps.lat", 1e-7, 0, "degrees times 1e7; 0 means no fix and is dropped"),
147      ("GPS_RAW_INT", "lon", "degE7", "gps.lon", 1e-7, 0, "degrees times 1e7; 0 means no fix and is dropped"),
148      ("GPS_RAW_INT", "alt", "mm", "gps.alt_msl", 0.001, 0, "millimetres to metres, above mean sea level"),
149      ("GPS_RAW_INT", "fix_type", "enum", "gps.fix_type", 1, 0, "the same MAVLink enum PX4 uses (3 is a 3D fix)"),
150      ("GPS_RAW_INT", "satellites_visible", "count", "gps.satellites", 1, 0, "satellites VISIBLE; PX4 reports satellites USED, so counts run slight
151      ("VFR_HUD", "groundspeed", "m/s", "gps.speed", 1, 0, "ground speed from the navigation filter; PX4 reports GNSS speed"),
152      ("LOCAL_POSITION_NED", "vx", "m", "position.x", 1, 0, "local north, relative to home"),
153      ("LOCAL_POSITION_NED", "vy", "m", "position.y", 1, 0, "local east, relative to home"),
154      ("LOCAL_POSITION_NED", "vz", "m", "position.z", 1, 0, "local down, relative to home"),
155      ("LOCAL_POSITION_NED", "vx", "m/s", "velocity.x", 1, 0, "north"),
```

Method 1: pipelines that survive failure



Job tables in Postgres

- Status, attempts, last error, lease time per item
- Claimed with `FOR UPDATE SKIP LOCKED`: workers run concurrently
- Idempotent, atomic per item, failure-isolated, stale leases reclaimed
- Grow by enqueueing items, never by changing code

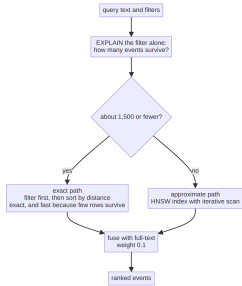
Method 2: hybrid search that routes itself

Hybrid retrieval

- Vector (HNSW) + full text (GIN) + SQL filters
- Fused by **reciprocal rank fusion**, weights (1.0, 0.1)
- With a 10% filter the planner chose HNSW post-filtering: **4 of 10 rows**, recall 0.37

The router

- Estimate filtered rows with `EXPLAIN` on the filter alone
- $\leq$ 1,500 rows: filter first, sort exactly; else HNSW with iterative scan



Method 3: guarded natural-language SQL



SQL guard: one SELECT, no forbidden statements, top-level WITH only

```
104     root = statements[0]
105     if not isinstance(root, (exp.Select, exp.SetOperation)):
106         raise GuardError(f"Only SELECT queries are allowed, got {type(root).__name__.upper()}.")
107
108     bad = next((n for n in root.walk() if isinstance(n, FORBIDDEN_NODES)), None)
109     if bad is not None:
110         raise GuardError(f"{type(bad).__name__.upper()} is not allowed in a query.")
111
112     withs = [n for n in root.walk() if isinstance(n, exp.With)]
113     if any(w is not root.args.get("with_") and w is not root.args.get("with") for w in withs):
114         raise GuardError("WITH is only allowed at the top level of the query.")
115     if any(w.args.get("recursive") for w in withs):
116         raise GuardError("Recursive WITH is not allowed.")
117     cte_names = {c.alias_or_name.lower() for c in root.find_all(exp.CTE)}
118
119     used_tables: list[str] = []
120     for t in root.find_all(exp.Table):
121         if not isinstance(t.this, exp.Identifier):
122             raise GuardError("Table-valued functions are not allowed in FROM.")
123         name, schema = t.name.lower(), (t.db or "").lower()
```

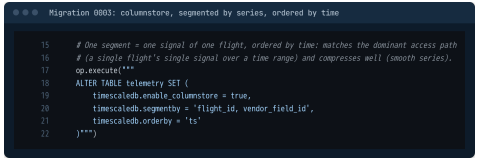
Four independent defences

- **Guard** (`sqlglot`): one SELECT, allow-lists, SQL rebuilt from the parse tree
- **Role** that reads only documented views
- **Read-only** transaction, 8 s timeout
- **Cost admission**: planner estimate under 100,000

Optimization: columnstore physical design

100 flights, 26.9 M rows, median of 7 warm runs; Q2 all flights, Q3 one flight, Q4 90 days

variant	size	Q2	Q3	Q4
rowstore + B-tree	3,631 MB	1,519 ms	65 ms	246 ms
segment (flight, signal)	**147 MB**	**16 ms**	10.8 ms	**2.7 ms**
no segmentby	157 MB	29.7 ms	13.7 ms	3.6 ms
1-day chunks	150 MB	15.3 ms	12.3 ms	3.4 ms
30-day chunks	143 MB	8.3 ms	5.7 ms	2.4 ms



Findings

- **25× smaller, up to 95× faster** cross-flight queries
- Design choices change size by under 10%
- Chunk count and `segmentby` change cross-flight speed about 2×
- Partial chunks must be recompressed: found by testing

Experimental setup

Environment

- 4 cores, 15 GB RAM, no GPU; everything in Docker
- PostgreSQL 17.11, TimescaleDB 2.30.2, pgvector 0.8.7, pgvectorscale 0.9.1, DuckDB 1.5.6
- `shared_buffers` 3,968 MB, `maintenance_work_mem` 1,500 MB

Method

- One connection, warm cache, 3 measured passes or median of 5–7 runs
- Recall vs exact search; 200 held-out queries; top 10
- Dev set and held-out set; bootstrap intervals and paired tests
- Every mistake and its fix is in `docs/decisions.md`

Results I: storage and engine comparison

70.7 M rows, size

representation	size	B/row
rowstore + index	12,459 MB	176.2
columnstore	**385 MB**	**5.4**
Parquet (zstd)	392 MB	5.5
DuckDB native	700 MB	9.9

Query p50 in ms: Timescale vs DuckDB

query	Timescale	DuckDB pq	DuckDB nat.
Q1 one signal, one flight	25.4	25.3	**4.9**
Q3 one flight, all signals	**18.7**	45.7	22.5
Q4 90-day window	**3.8**	28.4	4.5
Q5 all-data aggregate	811.8	1,057.8	**478.3**
Q6 per-flight summary	**1,007.5**	2,605.3	2,004.0

Q2 (one signal, all flights): 69.2 / 30.6 / 12.9. Timescale `COPY`: 287 k rows/s

Results II: vector indexes and filtered search

100,000 vectors, 1,536 dims (exact: 362 ms)

index	build	size	p50 at recall 0.95
HNSW m=16	56 s	819 MB	2.7 ms
HNSW m=32	149 s	819 MB	**2.4 ms**
IVFFlat 316	**17 s**	822 MB	5.5 ms
DiskANN	268 s	**68 MB**	not reached

Rows returned of 10, filter keeps. . .

strategy	0.1%	1%	10%
HNSW post-filter, ef 40	**0.0**	0.4	4.1
HNSW iterative scan	10	10	10
exact sort (B-tree)	10	10	10

Exact sort 1.0 ms at 0.1%, 55 ms at 10%; crossover near 1,000 rows

Results III: search quality and NL-to-SQL

Search, nDCG@10 (65 queries)

method	nDCG@10
vector only	**0.710**
vector + light full text	**0.709**
plain RRF	0.600
full text only	0.48

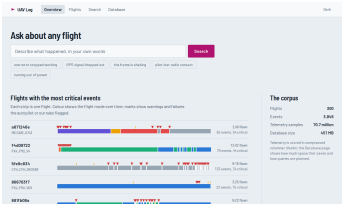
Plain fusion is significantly worse: paired -0.109, 95% CI $[-0.172, -0.051]$

NL-to-SQL, held-out (27 questions)

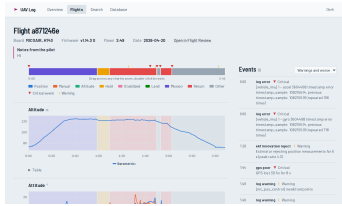
condition	strict EX
semantic views	**0.89**
raw tables + same docs	0.70
raw tables alone	0.26

Documentation is the big effect ($p = 0.0005$); views add 0.19, not significant ($p = 0.125$). 46 hostile queries refused.

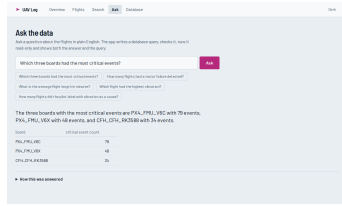
The interface



Explorer



One flight: mode ribbon, envelopes



Ask: SQL, cost, guard actions

Conclusions, limits and next steps

Conclusions

- One PostgreSQL holds time series, ontology, vectors and guarded NL-to-SQL
- Physical design cut storage **32**×; filtering strategy mattered more than index choice
- Documentation drives NL-to-SQL accuracy; four layers keep it safe
- Hybrid fusion did *not* help: reported as found

Limits and next steps

- 300 flights, one machine, one model, author-written questions
- Index-benchmark vectors are perturbed copies of real ones
- Next: the large run (10 k+ logs), ArduPilot at scale, concurrency benchmark
- Snapshot tool is untested end to end